

7-8-26

## Experiment - 21

Aim:

To implement the worst fit memory management algorithm in C.

Code:

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int b[5] = {100, 300, 200, 300, 600};
```

int P[2][5] = {2, 4, 1, 4, 2, 6};

int i, j, w;

printf("Process 1 & Block 1\n");

for (i = 0; i < 4; i++)

w = -1;

for (j = 0; j < 5; j++)

if (b[j] == P[i] && (w == -1 || b[j] == b[w]))

w = j;

if (w != -1)

printf("P1 - d | d | d | d | a", P[w]);

b[w] = P[i];

} else

printf("P1 - d is not allocated\n", i);

}

return 0;

}

Output:

| Process | Block |
|---------|-------|
| P1      | 5     |
| P2      | 2     |
| P3      | 5     |

P<sub>n</sub> not allocated.

Result: ~~First~~ First memory allocation algorithm  
was implemented and executed successfully.

7.8.26

Experiment - 22

Aim:

To implement the Best Fit memory management algorithm in C

Code:

```
#include <stdio.h>
```

```
int main (1 {
```

```
int b[5] = {100, 500, 200, 300, 600};
```

```
int p[n] = {212, 407, 112, 426};
```

```
int i, j, best;
```

```
printf ("Process \t Block \t n");
```

```
for (i = 0; i < 4; i++) {
```

```
best = -1;
```

```
for (j = 0; j < 5; j++)
```

```
if (b[j] >= p[i] && best == -1 ||  
b[j] < b[best]))
```

best = j;

if (best == -1) {

printf("Process %d not allocated\n", i+1);

b[best] = p[i];

}

printf("Process %d not allocated\n", i+1);

return 0;

}

output:

Process

Block

P1

P2

P3

P4

2

3

5

Result:

The Best Fit ~~algorithm~~ allocation algorithm was implemented and executed successfully.

10.08.2026

Expendiment

Ans:

→ implement the First Fit memory allocation algorithm using C.

Code:

```
#include <stdio.h>

int main() {
    int b[100], p[100], a[100];
    int m, n, i, j;

    printf("Enter number of blocks: ");
    scanf("%d", &m);

    printf("Enter block size: ");
    for(i=0; i<m; i++)
        scanf("%d", &b[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter process sizes: ");
    for(i=0; i<n; i++)
        scanf("%d", &p[i]);

    for(i=0; i<n; i++)
        scanf("%d", &p[i]);
}
```

Red

3.

Out for

Ent

Ent

En

E

1

F

Output:

Enter number of blocks: 3

Enter block sizes: 100 500 200

Enter number of processes: 3

Enter process sizes: 212 100 50

| Process                  | Size | Block          |
|--------------------------|------|----------------|
| P <sub>1</sub>           | 212  | B <sub>2</sub> |
| P <sub>2</sub>           | 100  | B <sub>1</sub> |
| <del>P<sub>3</sub></del> | 50   | B <sub>3</sub> |

~~Result~~  
Thus, the First Fit memory allocation algorithm was successfully implemented.

10.08.2026 Experiment - 24

Aim:

To write a C program to demonstrate UNIX system calls for file management such as open(), read(), write() and close().

Code:

```
#include <stdio.h>
#include <fcntl.h>
```

#include <unistd.h>  
int main () {

int fd;  
char buffd [100];

int n;

fd = open ("sample.txt", O\_CREAT | O\_RDWR, 0644);

if (fd == -1) {

printf ("File open error");

return 1;

}

write (fd, "Hello UNIX System calls", 23);

lseek (fd, 0, SEEK\_SET);

n = read (fd, buffd, (sizeof(buffd) - 1));

buffd[n] = '\0';

printf ("File Contents: \n %s", buffd);

close (fd);

return 0;

}

Output

File

hell

Rel

Co

10

Output:

File Content:

hello UNIX system calls.

Result:

Thus, the UNIX file management system calls `open()`, `write()`, `read()`, `seek()`, and `close()` were successfully demonstrated.

10.08.2026

Experiment - 25

Aim:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```
int main()
```

```
{
    int fd;
```

```
    struct stat st;
```

```
    DIR *d;
```

```
    struct dirent *e;
```

```
    fd = open("Sample.txt", O_CREAT | O_
    0644);
```

```
    printf("FD = %d\n", fcntl(fd, F_GETFL));
```

```
    write(fd, "UNIX calls", 10);
```

```
    seek(fd, 0, SEEK_SET);
```

12/20/23  
95  
1000 C  
JUL 12

v. 07

Nov

of

1

2

~~code~~ ~~Read~~, the UNIX FIO system calls `read()`, `write()`, `stat()`, `open()`, and `close()` were successfully demonstrated.

10.08.2026

Experiment - 26

Aim:

To implement basic file management operations such as ~~create~~, write, read, and closing using C.

Code:

```
#include <stdio.h>
int main () {
    FILE *fp;
    char data[100];
    fp = fopen ("sample.txt", "w");
    printf ("Enter data: ");
    fgets (data, sizeof(data), stdin);
    fputs (data, fp);
    fclose (fp);

    fp = fopen ("sample.txt", "r");
    printf ("File content: \n");
```

while (fgets(data, sizeof(data), fp))  
printf("File operation completed.");  
fclose(fp);  
printf("File operation completed.");  
return 0;

}

Output:

Enter data: Hello File Management.

File Content:

Hello File Management.

File operation completed.

Result:

~~Thus~~, the file management operations were successfully implemented in C.

12.08.2026

Experiment - 27

Aim:

To write a C program to simulate the ls UNIX Command and display the files in the current directory.

code:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
```

```
    DIR *dir;
```

```
    struct dirent *file;
```

```
    dir = opendir(".");
```

```
    if (dir == NULL) {
```

```
        printf("Unable to open directory\n");
```

```
        return 1;
```

```
    }
    printf("Files in current directory: \n");
```

```
    while ((file = readdir(dir)) != NULL) {
```

```
        printf("%s\n", file->d_name);
```

```
    }
    closedir(dir);
```

```
    return 0;
```

}

Output:

Files in current directory:

ON

Sample.txt

progname.c

a.out

Result:

Thus the ls UNIX command was  
successfully simulated using C.

Experiment - 28

12.08.2026

Aim:

To write a C program to simulate the  
CIPED UNIX command.

Code:

```
#include <stdio.h>
#include <string.h>
```

```
int main (1) {
```

```
FILE *fp;
```

```
char word[50], line[200];
```

```
fp = fopen ("sample.txt", "r");
```

```
if (fp == NULL) {
```

```
printf ("File not found\n");
```

```
return 1;
```

```
}
```

```
printf ("Enter word: ");
```

```
scanf ("%s", word);
```

if (strcmp(argv[1], "c") == 0) {  
 printf("C\n");  
}

}  
return 0;  
}

3  
Output:

Enclosed: UNIX

UNIX is an operating system

UNIX supports system calls.

Result:

Thus, the C program UNIX command was  
successfully simulated using C.

12.08.2026

Experiment - 29

Aim:

To write a C program to simulate a  
critical process synchronization problem using  
the producer-consumer problem.

Code:

#include <stdio.h>

int b[5], in=0, out=0, count=0;

```
void phrase (int i) {  
    if (count == 5) printf ("Buffer Full\n");
```

```
    else {
```

```
        b[in] = x;
```

```
        in = (in + 1) % 5;
```

```
        count++;
```

```
        printf ("Produced: %d\n", x);
```

```
    }
```

```
}
```

```
void consume () {
```

```
    if (count == 0) printf ("Buffer Empty\n");
```

```
    else {
```

```
        printf ("Consumed: %d\n", b[out]);
```

```
        out = (out + 1) % 5;
```

```
        count--;
```

```
    }
```

```
}
```

```
int main () {
```

```
    int ch, x;
```

```
    while (1) {
```

```
        printf ("\n1. Phrase 2. Consume)
```

```
3. Exit\n");
```

if (c == 'P')

if (c == 'P')

if (c == 'P')

if (c == 'P')

if (c == 'P')

if (c == 'P')

if (c == 'P')

if (c == 'P')

if (c == 'P')

if (c == 'P')

if (c == 'P')

if (c == 'P')

if (c == 'P')

if (c == 'P')

if (c == 'P')

Rebels:

The mutual-exclusion process.  
Synchronization problem was successfully  
solved using C.

17.08.2026

Experiment 30

Aim:

To demonstrate thread operations  
create, join, equal and exit using POSIX  
threads

Code:

```
#include <stdio.h>
#include <pthread.h>
void* fun (void*) {
printf ("Thread is running\n");
pthread_exit(NULL);
}
```

3

```
int main () {
```

```
pthread_t t1, t2;
```

```
pthread_create (&t1, NULL, fun, NULL);
```

```
pthread_create (&t2, NULL, fun, NULL);
```

if (pthread\_equal(t1, t2))  
printf("Threads are not equal\n");  
printf("Threads joined successfully\n");  
return 0;

3.

Input:

~~Thread is running~~  
~~Thread is running~~  
~~Threads are not equal~~  
~~Threads joined successfully.~~

Result:

~~Thread create, join, equal, and~~  
~~not operation were successfully demonstrated~~